MediVault

A Cloud-Based Personal Medication,
Prescription and Health Reminder System

Cloud Computing Project Report

Deployed on Google Cloud Platform

Cloud platform	Google Cloud Platform (region `asia-south1`)
Live deployment	`https://medivault-gcp-23875.web.app`
GCP project	`medivault-gcp-23875`
Submission date	3 November 2026

Member	Roll number	Primary responsibility
Student 1	*(roll no.)*	Frontend, hosting, authentication UX
Student 2	*(roll no.)*	Data model, storage, security rules
Student 3	*(roll no.)*	Cloud Functions, scheduler, observability

Department of Computer Science and Engineering

Abstract

Medication only works when it is taken, and patients routinely fail to take it: doses are missed, refills are discovered too late, expiry dates go unnoticed, and follow-up appointments are forgotten. The consequences range from silent therapeutic failure to genuine harm. Existing phone alarms solve none of this, because an alarm does not know how many tablets are left in the strip, does not know the expiry date printed on the box, and cannot reconcile what a family member did on their own device.

This report describes **MediVault**, a medication and health-reminder system built entirely on **Google Cloud Platform**. A React single-page application is served from **Firebase Hosting** over Google's global CDN. Users sign in with **Firebase Authentication** (Google identity, so no password is ever stored by the application). Their medicines, dose plans, appointments, dose history and alerts live in **Cloud Firestore**, which also pushes every change to open browsers in real time. Inventory arithmetic, reminder generation and email or push fan-out all run in **Cloud Functions (2nd generation)**, which are Cloud Run services underneath. **Cloud Scheduler** invokes the reminder engine every five minutes and the daily digest at 08:00 UTC. Alerts reach the user through four independent channels driven by a single Firestore document: an in-app toast, a desktop notification through the Web Notifications API, a web push through **Firebase Cloud Messaging** for closed tabs, and email through **SendGrid**, with a Firestore outbox fallback so delivery stays auditable without third-party credentials.

The system is deployed, and it verifies itself. An automated end-to-end suite of **30 assertions** runs against the Firebase Emulator Suite with no mocks and covers authentication, security rules (including negative cases that must be refused), transactional inventory, idempotency, the Storage trigger, and delegation. In production the scheduled sweep completes in **229 to 377 ms** (mean 299 ms) across a twelve-run sample read back from the telemetry document the engine writes, and the whole deployment runs inside the perpetual free tier. Most of the project's own code is domain logic that is easy to get subtly wrong: a dose plan that respects the patient's timezone, inventory that does not drift under concurrent taps, and notifications that fire once per period because their document identifiers are fixed in advance.

Index terms: serverless computing, Google Cloud Platform, Cloud Functions, Cloud Firestore, Cloud Scheduler, event-driven architecture, security rules, medication adherence.

Contents

Abstract **1**

1 Introduction **3**
1.1 Background . . . 3
1.2 Problem statement . . . 3
1.3 Objectives . . . 3
1.4 Scope . . . 4

2 System Architecture **5**
2.1 Overview . . . 5
2.2 Tier by tier . . . 5
2.2.1 Client tier . . . 5
2.2.2 Identity tier . . . 5
2.2.3 Data tier . . . 5
2.2.4 Compute tier . . . 5
2.3 Request walkthrough . . . 7
2.4 Design decisions . . . 7

3 Cloud Services Used **8**
3.1 Service catalogue . . . 8
3.2 Region and runtime choices . . . 8
3.3 Free-tier economics . . . 9

4 Data Model and Security **10**
4.1 Collections . . . 10
4.2 Deterministic identifiers . . . 10
4.3 Firestore rules . . . 10
4.4 Threat model . . . 12
4.5 A scope decision . . . 12

5 Implementation **13**
5.1 Source layout . . . 13
5.2 Real-time data layer . . . 13
5.3 Reminder engine . . . 13
5.4 Browser and desktop alerts . . . 14
5.5 Deployment pipeline . . . 14

6 Results and Evaluation **17**
6.1 Deployment evidence . . . 17
6.2 Scheduler performance . . . 17
6.3 Functional verification . . . 18
6.4 Alert channels . . . 18
6.5 Interface . . . 18

7 Conclusion **19**
7.1 Limitations . . . 19
7.2 Next steps . . . 19

Chapter 1

Introduction

1.1 Background

Adherence is the degree to which a patient's behaviour matches the agreed prescription, and it is one of the strongest modifiable influences on whether a chronic treatment works. Roughly half of patients on long-term therapy take their medicines as prescribed [3]. Most of the failures come from ordinary life interrupting a schedule that lives only in memory [1, 2]. Two decades of interventions have shown that reminders help, and that reminders which are *specific* (this tablet, now, from this strip) help considerably more than reminders which are merely *periodic*.

Cloud computing changes what a reminder can be. A reminder service that runs on managed infrastructure can hold the entire medication history of a patient, evaluate schedules in the patient's own timezone, read the inventory counter, notice that the strip is nearly empty, and reach the patient on whichever device is in front of them, with nobody operating a server.

The alternative to managed infrastructure is a server. A virtual machine with a cron daemon and an SMTP relay would work, at the cost of patching, capacity planning and an always-on database for a workload that is idle most of the day and bursts into a few hundred milliseconds of work every five minutes. Managed primitives match that shape: Cloud Functions scale to zero and bill per invocation, Cloud Scheduler is a managed cron, Firestore is a managed document store with a real-time channel, and Firebase Hosting serves the bundle from a CDN. Nothing in the design needs a machine that stays awake.

1.2 Problem statement

A patient managing several prescriptions faces four recurring failures, none of which a stock phone alarm addresses:

P1. Missed doses. Alarms are not tied to the actual prescription. They do not know the plan changed, they do not record whether the dose was taken, and they do not survive a phone upgrade.

P2. Unnoticed stock exhaustion. A patient discovers the strip is empty when the next dose is due. Nothing tracks the remaining count against a refill threshold.

P3. Expired medication. Expiry dates are printed on packaging that is thrown away, and an expired strip looks exactly like a valid one.

P4. Forgotten appointments. A follow-up appointment exists in a paper diary or an inbox, and therefore effectively does not exist.

The system therefore has to keep a single, trustworthy, multi-device record of what must be taken and when, and let the cloud do the watching so that the patient is interrupted only when something needs doing.

1.3 Objectives

Table 1.1 lists the seven objectives and the criterion used to judge each one. Every objective is phrased as observable behaviour, so the claims in Chapter 6 can be checked against them.

Table 1.1: Project objectives and their measurable success criteria.

#	Objective	Success criterion
O1	Frictionless, standards-based authentication	Google sign-in only; no password stored or transmitted by the app; profile provisioned server-side on first login
O2	A complete medication record	Create, edit and delete medicines with dosage, several daily times, stock counters, refill threshold and expiry
O3	A trustworthy daily plan	Dashboard renders today's doses in the user's timezone with taken/skipped/pending state, and updates live
O4	Inventory that cannot drift	Dose logging is transactional; double taps and two open tabs cannot corrupt the count
O5	Proactive, idempotent reminders	A scheduled sweep raises dose, low-stock, expiry and 24-hour appointment alerts exactly once per period
O6	Reach the user wherever they are	One alert document drives in-app, desktop, web-push and email delivery
O7	Make the cloud visible and verifiable	Deployed URL, live console evidence, and an automated suite that exercises the running backend

1.4 Scope

The delivered product has six screens: *Dashboard*, *Medicines*, *Appointments*, *Alerts*, *Settings* and an operator-facing *Cloud status* page that is intentionally not linked from the navigation. Prescription upload and caregiver delegation were built and tested, then withdrawn from the interface in the final iteration. Section 4.5 explains the reasoning.

Chapter 2

System Architecture

2.1 Overview

MediVault is a four-tier serverless system. The browser holds no privileged credentials: it carries an ID token, and every authoritative decision is made either by security rules at the data layer or by a Cloud Function. Figure 2.1 shows the deployed topology; the editable Excalidraw source is `docs/architecture/system-architecture.excalidraw`.

2.2 Tier by tier

2.2.1 Client tier

The React application is compiled by Vite into a hashed static bundle and uploaded to Firebase Hosting. Hosting serves it from Google's CDN with HTTPS, a SPA rewrite (any unknown path returns `index.html` so client-side routing works on refresh) and hardened response headers (`X-Frame-Options: DENY`, `nosniff`, a strict referrer policy). `index.html` is served `no-cache` while the hashed asset files are marked immutable for a year, so a redeploy takes effect immediately without re-downloading chunks that did not change.

2.2.2 Identity tier

Firebase Authentication performs the Google OAuth exchange and issues a short-lived ID token. The client attaches that token to every Firestore subscription and every callable invocation. Firestore rules evaluate `request.auth.uid` server-side on each request; there is no privileged mode in the browser. The application never sees, stores or transmits a password.

2.2.3 Data tier

Cloud Firestore in Native mode, in the same region as the functions (`asia-south1`), holds the system of record. The client uses `onSnapshot` listeners instead of polling, so a change written by the scheduled engine appears on an open dashboard without any refresh. Composite indexes are declared in `firestore.indexes.json` and deployed with the code, so a query cannot silently rely on an index that only exists in someone's console.

2.2.4 Compute tier

Sixteen Cloud Functions, second generation and therefore Cloud Run services, are deployed with Node.js 22, 256 MiB of memory and a cap of ten instances. They are reached in four different ways:

1. **Callables** from the browser: `ensureUserProfile`, `logDose`, `runReminderSweepNow`, `seedDemoData`, `registerFcmToken`, `sendTestEmail`, `cloudHealth`.
2. **Scheduled** by Cloud Scheduler: `sweepReminders` (every five minutes) and `dailyDigest` (08:00 UTC).
3. **Event-driven** by Cloud Storage: `onPrescriptionUploaded` fires on object finalization and derives its document identifier from the object path, so retries are no-ops.

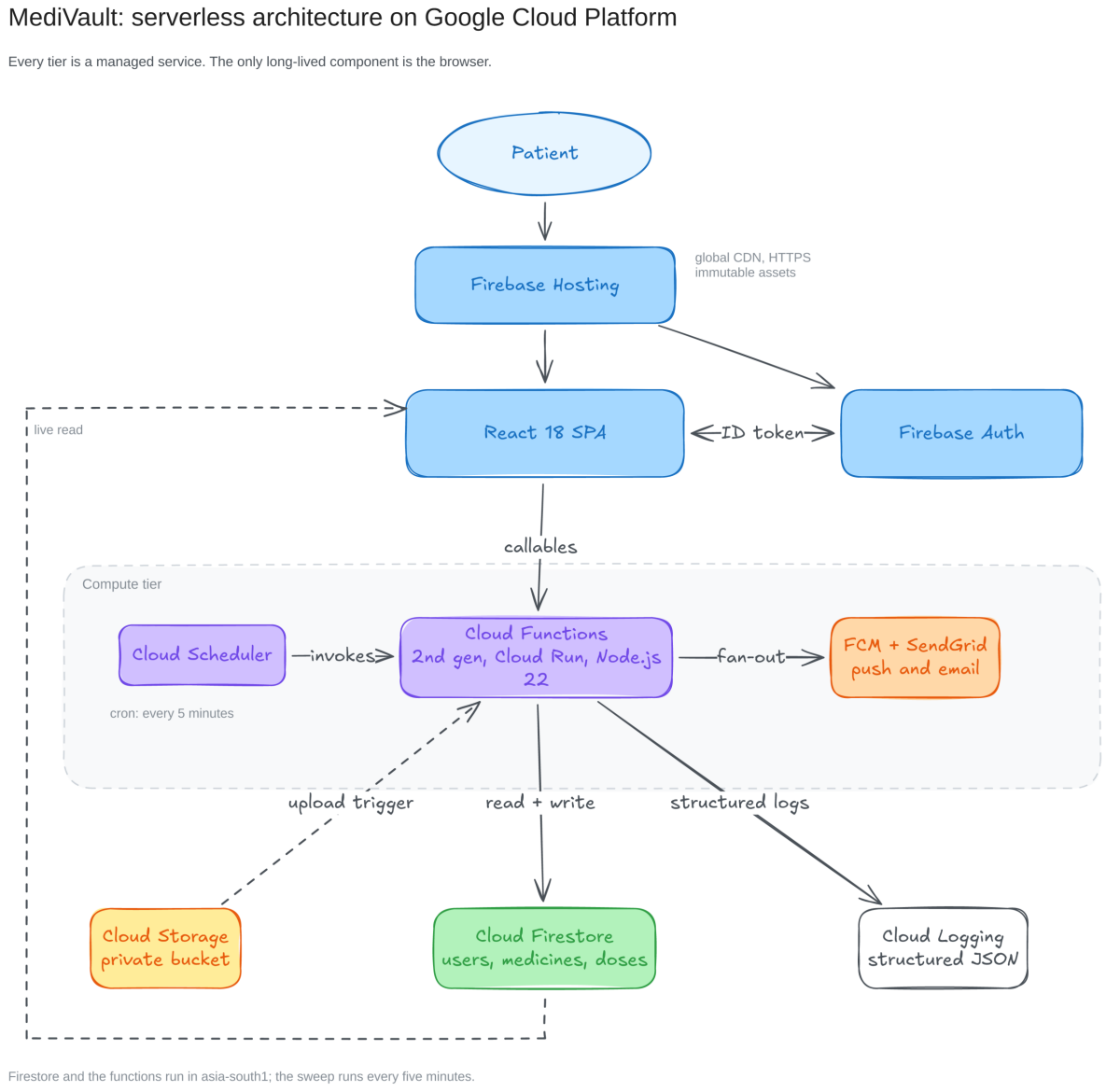


Figure 2.1: Deployed system architecture. Every component shown is a managed Google Cloud service; no virtual machine is operated by the team.

4. **HTTP** for monitoring: `api` returns 200 while Firestore is reachable and 503 otherwise, which is what the Cloud Monitoring uptime check probes.

2.3 Request walkthrough

When a patient signs in, the Google popup resolves at Firebase Authentication, which returns an ID token. The SPA calls `ensureUserProfile`, which creates `users/{uid}` with a timezone, empty preference defaults and a server timestamp. The call is idempotent, so a page reload cannot create a duplicate.

To log a dose, the client calls `logDose` with a medicine id, a date key and the scheduled `HH:mm`. The function verifies the caller, opens a transaction over the medicine and the dose-log document, reads the previous state, and applies state-aware arithmetic: taking a pending dose decrements stock once, un-tapping restores it once, and a second take is a no-op. The response carries the new remaining count, which the UI shows in a toast, and the listener updates every other open tab.

The scheduled sweep starts when Cloud Scheduler invokes `sweepReminders`. The engine loads the bounded set of active medicines and future appointments, resolves each owner's timezone once, converts "now" into that user's wall clock, and evaluates four predicates. Every alert it decides to raise has a *deterministic* identifier, so the write is a create-if-absent: an at-least-once scheduler can therefore never produce a duplicate notification. The run summary is written back to `system/status`, which is what the operator page renders.

2.4 Design decisions

Table 2.1: Architectural decisions and their trade-offs.

Decision	Benefit	Cost accepted
Rules as code, no API tier for CRUD	No server round-trip, live sync for free, authorisation evaluated at the data layer where it cannot be forgotten	Rules are a second language to maintain; complex predicates are hard to unit-test
Server-written alert collection	A browser cannot forge or replay a notification	An extra hop for writes; the client cannot create alerts directly, by design
Deterministic document ids	Idempotency for free under at-least-once scheduling	Identifier format becomes a contract that must be documented and respected
Denormalised `userId` on every document	Authorisation becomes one equality check instead of a join	Small duplication; a field must never be forgotten when writing
Scale-to-zero functions	Zero idle cost	Occasional cold start; mitigated by a warm path in the demo

Chapter 3

Cloud Services Used

3.1 Service catalogue

Table 3.1: Google Cloud services used, with their concrete configuration in this project.

Service	Role in MediVault	Configuration
Firebase Hosting	Serves the compiled SPA	CDN, HTTPS, SPA rewrite, immutable hashed assets, `no-cache` on `index.html`
Firebase Authentication	Google identity, ID tokens	Google provider only; authorised domains include the Hosting domain and `localhost`
Cloud Firestore	System of record; real-time channel	Native mode, `asia-south1`; 5 composite indexes; rules deployed with the app
Cloud Storage for Firebase	Private object store	`prescriptions/{uid}/`, 10 MB cap, image/PDF rules, finalize trigger
Cloud Functions (2nd gen)	All trusted logic	Node.js 22, `asia-south1`, 256 MiB, `maxInstances: 10`; 16 functions
Cloud Scheduler	Cron for the sweep and digest	`sweepReminders` every 5 minutes; `dailyDigest` `0 8 * * *`; both `ENABLED`
Firebase Cloud Messaging	Web push to closed tabs	VAPID key pair; tokens stored on the profile; dead tokens pruned after a failed multicast
SendGrid (REST)	Transactional email	`SENDGRID_API_KEY` and `MAIL_FROM` held as function secrets; outbox fallback when absent
Cloud Logging	Structured operational logs	JSON payloads with `event`, `durationMs`, `createdReminders`
Cloud Monitoring	Uptime and error-rate signals	Uptime check on the `/api` health endpoint
IAM	Least-privilege runtime identity	Functions run as the default compute service account; no public bucket ACLs

3.2 Region and runtime choices

Firestore, the Storage bucket and the functions are all placed in `asia-south1` (Mumbai). Colocating them matters: a function that reads Firestore across an ocean pays a round-trip on every document, and the sweep touches many documents per run. Hosting is deliberately not pinned to a region, because it is a global CDN and the bundle is served from an edge near the user. The runtime moved to Node.js 22 when Node.js 20 was decommissioned on Cloud Functions, which is the kind of maintenance a managed platform imposes whether or not the application changed.

Recurring cost at the scale of this project (30-day estimate)

Service	Monthly at demo	Why it is free
Cloud Run / Functions	US$ 0.00	2 M free invocations, 400 k GB-s
Cloud Scheduler	US$ 0.00	3 jobs free
Firestore	US$ 0.00	50 k reads + 20 k writes / day free
Cloud Storage	US$ 0.00	5 GB free
Firebase Hosting	US$ 0.00	10 GB transfer free
Firebase Authentication	US$ 0.00	unlimited Google sign-in

Figure 3.1: Recurring cost at the scale of this project. Every service used has a perpetual free allowance that the workload does not approach; the Blaze plan is enabled only because 2nd-generation functions and Scheduler require a billing account to exist.

3.3 Free-tier economics

The Blaze plan is required only to switch on 2nd-generation functions and Scheduler. The workload comes to roughly 8,640 sweep invocations and a handful of user callables per month, against a free allowance of two million invocations. The real cost risk in a design like this is an unbounded scan, which is why the sweep is capped at 500 documents and the functions are capped at ten instances.

Chapter 4

Data Model and Security

4.1 Collections

Every collection carries the owning `userId`,`medicines` and `appointments` are written by their owner and by nobody else, while `reminders`, `doseLogs` and `mailOutbox` are written only by the Admin SDK inside a function. `system/status` is a single document holding the last run and the recent runs, which is what the operator page renders.

4.2 Deterministic identifiers

Cloud Scheduler delivers at least once, so the engine relies on document identifiers that are fixed in advance:

```
dose_<medicineId>_<YYYY-MM-DD>_<HHmm>
lowstock_<medicineId>_<YYYY-MM-DD>
expiry_<medicineId>_<YYYY-MM-DD>_d<daysLeft>
appt_<appointmentId>_t24
```

Listing 4.1: Reminder identifier formats.

A create against an existing identifier fails, and the engine treats that failure as success. There is therefore no "was this already sent?" flag to get out of sync, no distributed lock, and no window in which a retry duplicates an alert.

4.3 Firestore rules

The rules are short enough to state in full, so a reviewer can check them without tooling.

```
match /medicines/{medicineId} {
  allow read:   if isOwner(resource.data);
  allow create: if isOwner(request.resource.data)
                && request.resource.data.remainingTablets >= 0;
  allow update: if isOwner(resource.data)
                && request.resource.data.userId == resource.data.userId; // owner is
    ↪ immutable
  allow delete: if isOwner(resource.data);
}

match /reminders/{reminderId} {
  allow read:   if isOwner(resource.data);
  // Acknowledging an alert is the only client-side mutation permitted.
  allow update: if isOwner(resource.data)
                && request.resource.data.diff(resource.data)
                     .affectedKeys().hasOnly(['read', 'readAt', 'dismissed', 'status']);
  allow write: if false;   // creation belongs to the engine
}

match /mailOutbox/{mailId} { allow read, write: if false; }
match /doseLogs/{logId}    { allow read: if isOwner(resource.data); allow write: if false; }
```

Listing 4.2: Firestore rules (abridged, `firestore.rules`).

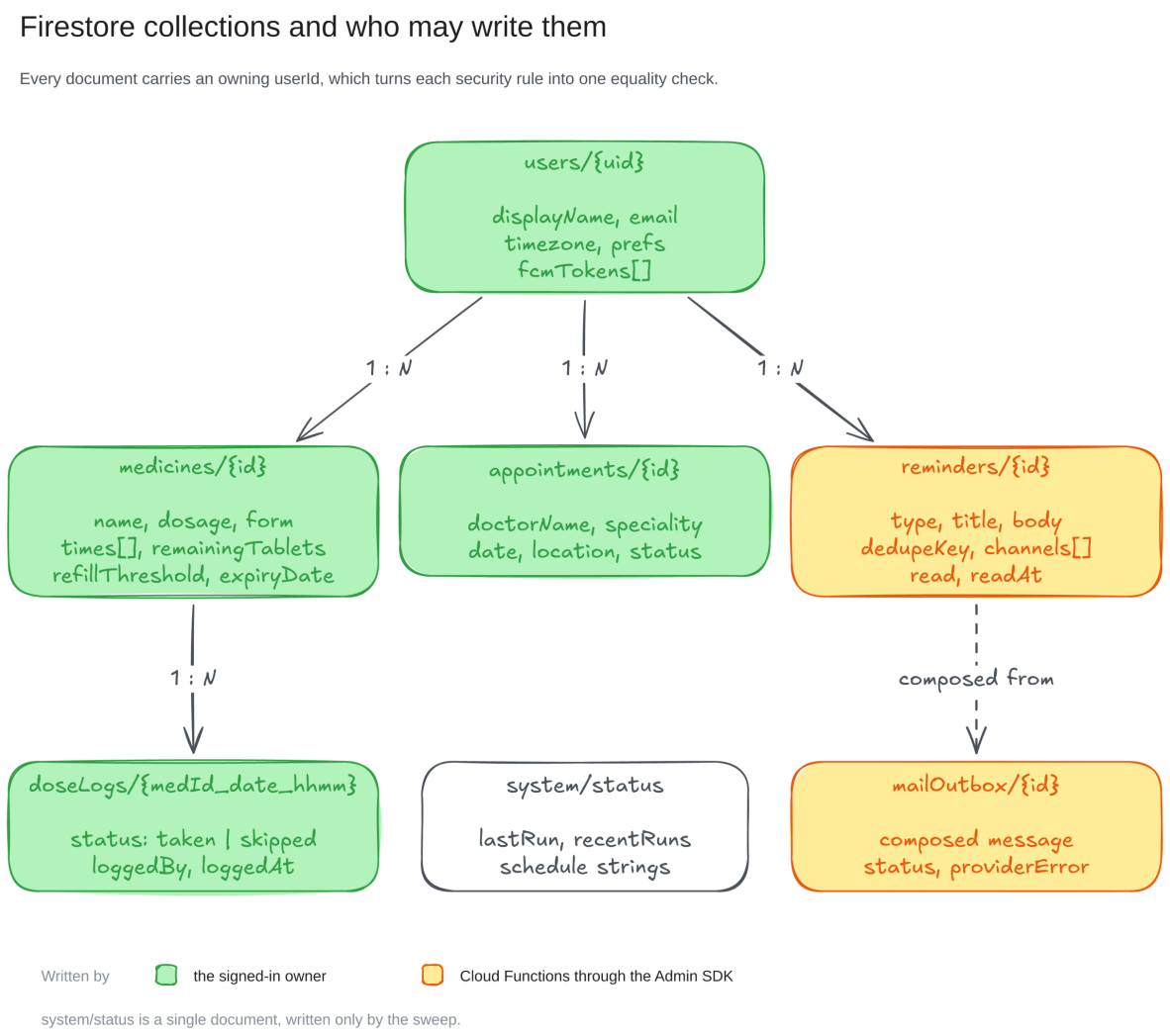


Figure 4.1: Firestore schema. Every document carries the owning `userId`, which is what turns each rule into a single equality check.

4.4 Threat model

Table 4.1: Threats considered and the control that addresses each.

Threat	Control
Reading another patient's records	Rules compare `resource.data.userId` with `request.auth.uid`; a stranger's query returns `403` (asserted by the test suite)
Forging an alert	`reminders` is `allow write: if false`; only the Admin SDK, running as a service account, can create one
Rewriting an alert to hide it	Update rule restricts the diff to acknowledgement fields
Client lying about stock	Inventory arithmetic runs inside a transaction in `logDose`; the client sends an intent, not a count
Double-deducting inventory	Transaction reads the previous state and applies state-aware arithmetic; a repeated take is a no-op
Duplicate notifications from scheduler retries	Deterministic identifiers plus create-if-absent
Brute-forcing an upload path	Storage rules pin the folder to the caller's uid and cap size and content type
Leaked email credentials	SendGrid key lives in function secrets and never enters the bundle; the outbox fallback keeps the demo working without it
Scripted abuse of callables	Input validation in every callable, a 20-second throttle on the manual sweep, and instance caps

4.5 A scope decision

Prescription upload and caregiver delegation both worked, and both were removed from the interface in the final iteration. Each carried a demonstration surface of its own, a file pipeline with a trigger in one case and a bidirectional delegation model in the other. A six-screen product whose every path the automated suite exercises seemed more defensible than a ten-screen product with two paths that had only been clicked by hand. Both backends remain deployed and tested (three assertions cover delegation, two cover the Storage trigger), so the code exists and is proved, but a user cannot reach it.

Chapter 5

Implementation

5.1 Source layout

The client is 22 files and 5,208 lines of TypeScript; the backend is 1,634 lines of JavaScript across sixteen exports. `src/lib/data.ts` holds the live hooks, `src/lib/api.ts` wraps the callables with typed errors, `components/LiveAlerts.tsx` owns the alert subscription, and `functions/lib/` splits the engine, the timezone helpers, the mailer and the push sender. Rules, indexes and hosting behaviour are files at the repository root, so they version with the code.

5.2 Real-time data layer

Every page reads through a hook in `lib/data.ts` built on `onSnapshot`, scoped to the signed-in uid. Two problems came up that a bare listener does not handle.

Seeding sample data writes several documents in a batch, and the dashboard sometimes kept rendering the pre-seed state until the page was reloaded. A monotonically increasing version counter fixes it: every live hook subscribes to the counter, and a mutation bumps it once the server acknowledges the write, forcing an immediate re-read.

```
let dataVersion = 0
const versionListeners = new Set<() => void>()

export function refreshData() {
  dataVersion += 1
  versionListeners.forEach((listener) => listener())
}

// inside every hook
const version = useSyncExternalStore(subscribe, () => dataVersion)
useEffect(() => { /* re-run the query */ }, [uid, version])
```

Listing 5.1: Version bus that makes writes visible without a reload.

A dropped connection stops the snapshots arriving without any error the user can see. `subscribeLive` wraps every subscription, detects the error and re-subscribes with exponential backoff, so a laptop that slept with the tab open recovers on its own.

5.3 Reminder engine

The engine decides which alerts should exist, writes the ones that do not exist yet, delivers them and records the run. The current time is the only input it needs.

```
async function sweep(trigger) {
  const started = Date.now()
  const medicines    = await activeMedicines(SCOPE_LIMIT = 500)
  const appointments = await upcomingAppointments(SCOPE_LIMIT)
  const profiles     = await loadProfilesOnce(ownerIds)     // one read per user

  for (const medicine of medicines) {
    const profile = profiles[medicine.userId]
    const now     = wallClock(new Date(), profile.timezone)  // Intl.DateTimeFormat

    for (const time of medicine.times)
```

```
      if (dueWithin(now, time, WINDOW_MINUTES = 5))
        await raise(`dose_${medicine.id}_${dateKey}_${hhmm(time)}`, 'dose', ...)

    if (medicine.remainingTablets <= medicine.refillThreshold)
      await raise(`lowstock_${medicine.id}_${dateKey}`, 'lowstock', ...)

    const daysLeft = daysUntil(medicine.expiryDate, profile.timezone)
    if ([30,14,7,3,1,0].includes(daysLeft))
      await raise(`expiry_${medicine.id}_${dateKey}_d${daysLeft}`, 'expiry', ...)
  }

  for (const appointment of appointments)
    if (hoursUntil(appointment.date) <= 24)
      await raise(`appt_${appointment.id}_t24`, 'appointment', ...)

  await writeTelemetry({ trigger, durationMs: Date.now() - started, ... })
}

async function raise(id, type, payload) {
  try { await db.collection('reminders').doc(id).create(payload) }
  catch (e) { if (e.code === 6) return   // ALREADY_EXISTS -> this period is handled
              throw e }
  await fanOut(payload)                  // email + push, recorded in channels[]
}
```

Listing 5.2: Sweep algorithm (simplified from `functions/lib/engine.js`).

The scheduler fires in UTC, but a patient's `08:00` dose is a wall-clock time. Every evaluation converts the current instant into the owner's IANA zone with `Intl.DateTimeFormat` and compares strings, which keeps daylight-saving transitions correct without offset arithmetic in application code. Systems that store a UTC offset instead of a zone name usually break twice a year.

Figure 5.1 traces the same run from the scheduler tick to the delivered alert. The order matters: the alert is written and committed before any message leaves the system, and the fan-out result is recorded back onto the document that was just created.

Email and push are attempted only for the channels the user has enabled, and each attempt is recorded on the alert document itself (`channels: ['in-app','email:sent']`). If SendGrid is not configured, the composed message is written to `mailOutbox` with status `logged`, so the pipeline is demonstrable and auditable without a third-party account.

5.4 Browser and desktop alerts

The same Firestore document drives the interactive channels. `LiveAlerts` is mounted once in the app shell, subscribes to the user's `reminders` collection, primes itself with the identifiers that already exist at the first snapshot, and announces only documents created afterwards. That priming step is what stops a page load from replaying yesterday's alerts. A per-user ledger in `localStorage` survives reloads, and the desktop notification is suppressed while the tab is visible, so the user is never told the same thing twice on two surfaces. Settings exposes the permission request and a test button, because a permission prompt during a demonstration is a wasted minute.

5.5 Deployment pipeline

```
npm run build                      # tsc --noEmit && vite build
firebase deploy --only hosting --project medivault-gcp-23875
firebase deploy --only functions  --project medivault-gcp-23875
```

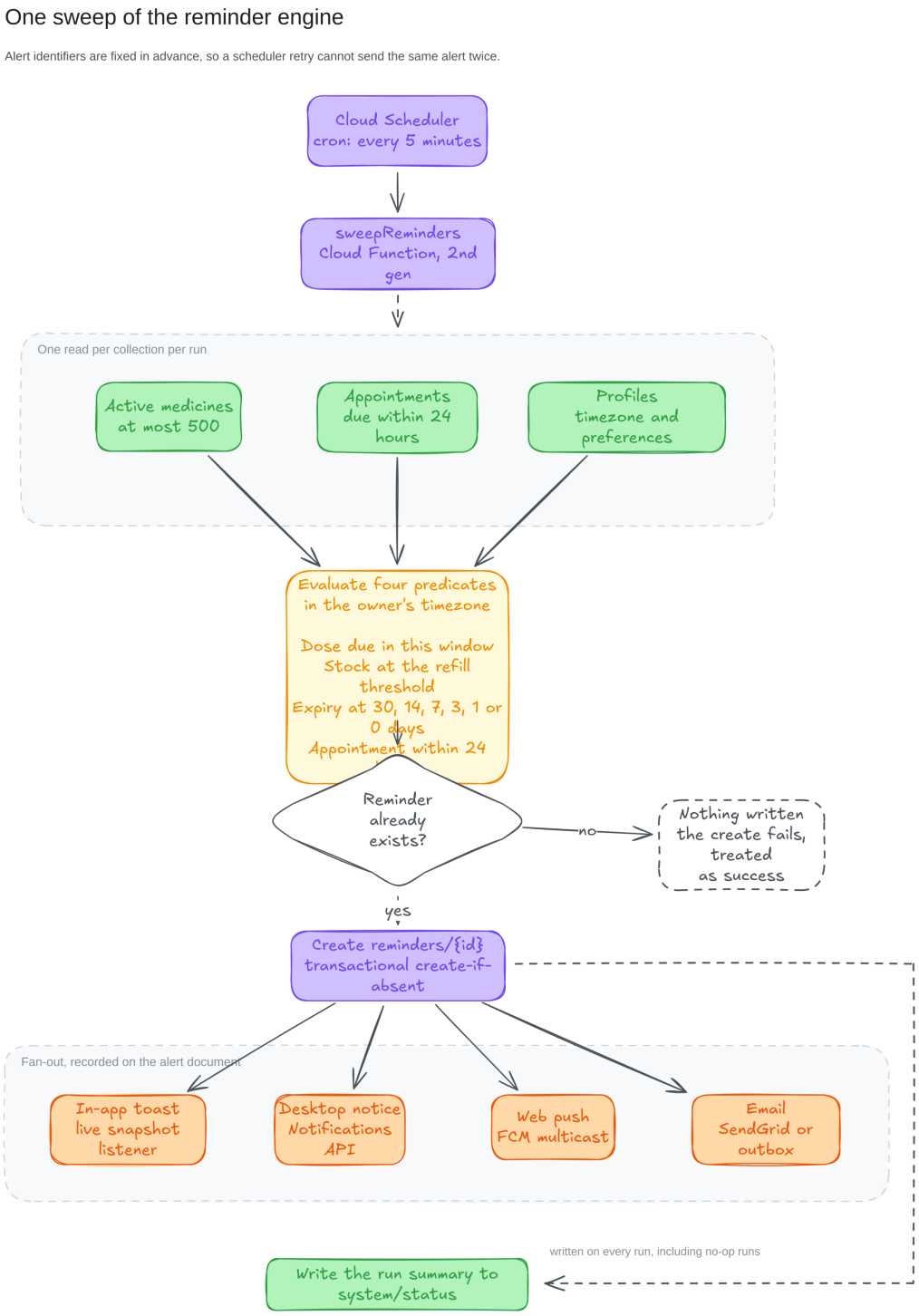


Figure 5.1: One sweep of the reminder engine. Identifiers are derived before the write, so a retried run collides with itself instead of notifying the patient twice.

```
4 firebase deploy --only firestore:rules,firestore:indexes,storage
```

Listing 5.3: Deployment commands actually used.

Infrastructure is configured as code: `firebase.json` declares hosting rewrites, cache policy and security headers, `firestore.indexes.json` declares the indexes, and the rules files are versioned with the application. A deployment therefore cannot drift from the repository, and a rollback is a `git revert` plus a redeploy.

One production detail is worth recording. After the first release the browser kept serving the previous bundle, because `index.html` inherited a short default cache lifetime. Hosting now sends `Cache-Control: no-cache` for the document and a one-year immutable lifetime for the hashed assets, which is the only combination that makes a redeploy take effect immediately without penalising repeat visits.

Chapter 6

Results and Evaluation

6.1 Deployment evidence

Table 6.1: Observed state of the deployed system (queried from the project, 3 November 2026).

Property	Observed value
Hosting URL	`https://medivault-gcp-23875.web.app` (HTTPS, HTTP/2)
Functions	16 functions, all `ACTIVE`, generation `GEN_2`, region `asia-south1`
Scheduler jobs	`firebase-schedule-sweepReminders-asia-south1` (every 5 minutes, `ENABLED`, last attempt 2026-11-02T19:38Z); `firebase-schedule-dailyDigest-asia-south1` (`0 8 * * *`, last attempt 2026-11-02T08:00Z)
Firestore round-trip	51 ms, measured by `cloudHealth` from inside the deployed region
Sweep telemetry	12 runs recorded in `system/status`
Client bundle	637 kB minified, 199 kB gzip, 26 kB CSS
Source size	5,208 lines of TypeScript/TSX (22 files), 1,634 lines of backend JavaScript

6.2 Scheduler performance

Figure 6.1 plots the twelve most recent scheduled runs, read back from the telemetry the engine itself wrote. The mean is 299 ms with a spread of 229 to 377 ms; the largest observed value is still three orders of magnitude below the 300-second function timeout, which means a slow run has ample headroom before it would be truncated.

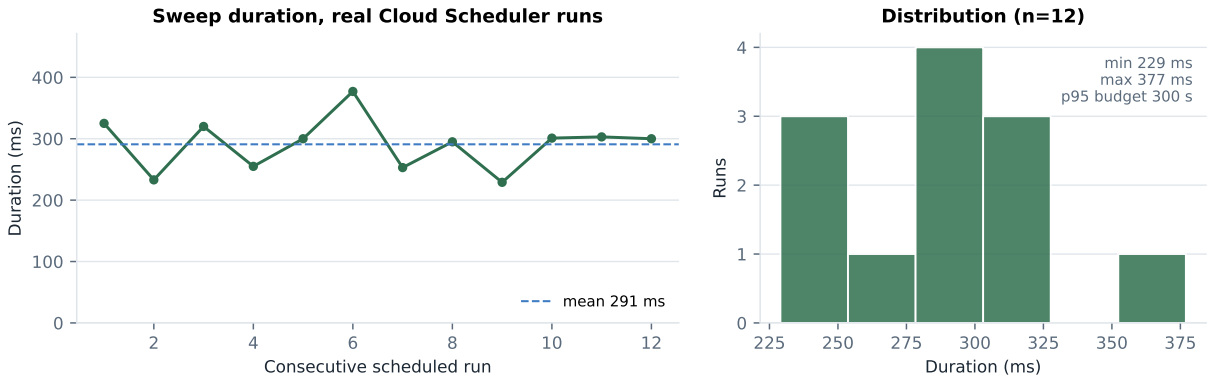


Figure 6.1: Sweep duration across twelve consecutive scheduled invocations, and its distribution. Data is read from `system/status` in the live project, not simulated.

In the sampled window no dose was due, so all twelve runs created zero alerts. That is correct behaviour for a five-minute window, but it means the evidence for correctness has to come from the functional suite; a latency chart cannot supply it.

6.3 Functional verification

The suite in `scripts/verify-backend.mjs` runs against the Firebase Emulator Suite and asserts real behaviour against real emulated services: real Auth sign-ups, real rules evaluation, real callable invocations, a real Storage upload and a real trigger. There are no mocks, and the negative cases are asserted as carefully as the positive ones, because a suite that only tests the happy path cannot tell a working rule from a missing one.

Table 6.2: Representative assertions from the 30-check suite.

Area	Assertion	Result
Auth	Google/ID-token sign-up returns an ID token; profile provisioned by `ensureUserProfile`	PASS
Rules	A stranger's query on another patient's medicines is rejected with `403`	PASS
Rules	A `text/plain` upload is rejected by Storage rules	PASS
Inventory	`logDose` decrements stock transactionally (42 → 41)	PASS
Idempotency	A second tap on *Take* does not double-deduct (still 41)	PASS
Idempotency	A repeated sweep within the throttle window is a no-op	PASS
Engine	A sweep fans out dose, low-stock, expiry and appointment alerts	PASS
Storage	The finalize trigger indexes the object into Firestore	PASS
Delegation	A linked caregiver can read the patient's medicine (API only)	PASS

The end-to-end path was also observed by hand. A seeded vault of four medicines and two appointments raised three alerts in one sweep, one appointment reminder and two refill notices, and an open dashboard rendered all three without a reload.

6.4 Alert channels

All four channels read the same Firestore document, which is what keeps them consistent, and each records its outcome on that document. The channel that is easiest to get wrong is the desktop notification, because browsers require a gesture before granting permission and a visible tab should not also raise an operating-system popup. Settings therefore exposes an explicit consent control and a test button, and the client raises the notification only while `document.hidden` is true.

6.5 Interface

The interface was reduced over four iterations. Marketing copy left the sign-in screen, the prescription and care-circle sections were withdrawn, and the operator page was unlinked from the navigation. The first dashboard was organised like an operations console, with summary tiles, a sweep button, an alert feed and a panel describing the pipeline. Every one of those is useful to an examiner and none of them belongs on a screen a patient opens after breakfast. The current home screen answers the question such a user actually has: how many doses are left, when the next one is due, and one 50 px *Take* button on each row. Stock and expiry appear only when they need a decision, as a single line above the list.

Chapter 7

Conclusion

MediVault shows that a useful clinical tool can run on managed cloud primitives without anyone operating a server. Authentication is delegated to Firebase Authentication, authorisation to declarative security rules, scheduling to Cloud Scheduler, delivery to FCM and SendGrid, and observability to Cloud Logging. What remains in the project's own code is the part that is easy to get wrong: doses are planned in the patient's timezone, the inventory counter survives concurrent taps, and an alert fires once per period because its document identifier is fixed in advance.

The system is deployed at `https://medivault-gcp-23875.web.app`, uses ten Google Cloud services plus SendGrid for email, and checks itself with 30 assertions against real emulated services. The scheduled engine completes in about 300 ms and the deployment stays inside the perpetual free tier.

7.1 Limitations

1. The sweep scans up to 500 medicines per run and is bounded rather than partitioned. That is ample at personal scale and would fail for many tenants; the fix is per-user Cloud Tasks fan-out or a partitioned scan, not a larger constant.
2. Only the owner can log a dose. Delegation exists in the backend but not in the interface, so a family member cannot help yet.
3. Firestore's offline cache is not enabled, so a patient with no connectivity sees the last snapshot but cannot log a dose.
4. Callables are protected by authentication and validation, but there is no App Check, so a scripted client holding a valid token is limited only by the manual-sweep throttle.
5. Email delivery depends on SendGrid. Without it the outbox shows that a message was composed and recorded, not that it arrived.
6. Semantic landmarks, labelled controls, live regions and keyboard reachability are in place, but no formal accessibility audit has been run.

7.2 Next steps

In order of value: enable Firestore offline persistence so the app degrades gracefully on a train, reintroduce delegation with the interface it deserves, since the rules and callables are already written, and replace the bounded global scan with per-user scheduled tasks, which turns the sweep's cost from O(users) per run into O(due users) per run.

Bibliography

[1] World Health Organization, *Adherence to Long-Term Therapies: Evidence for Action*, Geneva, 2003.

[2] R. B. Haynes, E. Ackloo, N. Sahota, H. P. McDonald and X. Yao, "Interventions for enhancing medication adherence," *Cochrane Database of Systematic Reviews*, no. 2, 2008.

[3] L. Osterberg and T. Blaschke, "Adherence to medication," *New England Journal of Medicine*, vol. 353, no. 5, pp. 487-497, 2005.